Customer Feedback & Business Improvement Agent

A Multi-Agent AI System for UrbanMart Customer Experience / Management

Company: UrbanMart

Tenzin Dolker Sherpa

Table of Contents

Overview 1
Part A: Understand the Business Problem 2
  Problem Identification 2
  Current Process (AS-IS Workflow) 3
  Process Analysis 4
  AI Opportunity 5
Part B: Multi-Agent System Design 6
  Agent Design 6
  Why Multiple Agents? 9
  Workflow Design (TO-BE) 10
  Human Review 11
Part C: Feedback Analysis & Business Logic 12
  Classification Approach 12
  Trend Identification 13
  Mixed or Unclear Feedback 14
  Business Recommendations 15
  Evidence 16
Part D: Build the Working System 17
  Platform and tool choices 17
Part E: Testing & Validation 21
  Testing Records 21
  Additional defects found during build 23
Part F: Final System Output 24
Part G: Business Review 25
  Business Impact 25
  Limitations 26
  Future Improvement 27
Final Presentation 28

Overview

UrbanMart is able to get customer feedback from surveys in the store, online surveys, email, its website and mobile app. All of the employees read all of the comments and prepare a summary for the management. This becomes slow, inconsistent, and difficult to identify the problems that keep repeating, the good experiences to be enjoyed, and what to try first.

This project designs, builds and tests a six-agent AI workflow in n8n that ingests feedback (from a built-in dataset loader and a live Google Sheet), classifies every comment against UrbanMart's official category, sentiment and severity references, separates genuine recurring patterns from isolated comments using deterministic counting, surfaces emerging and unusual issues, drafts practical recommendations grounded in the business context, checks those recommendations for evidence integrity, and compiles a management report that is emailed for human approval before any change is entered into the system and also the report response is also recorded in the status sheet.

The system was tested with the entire set of 30 records with mixed, vague, unusual, and changing-trend records. While testing, a number of defects were discovered and corrected:

1) A merge node that silently dropped live rows
2) A normalizer that didn't return anything
3) A fan-in that executed before the second analysis agent
4) Pacing wait node that did not wait.

The final system fulfills all the minimum requirements and retains all operational decisions to management.

Part A: Understand the Business Problem

Problem Identification

The problem UrbanMart has is that customer feedback comes in from 5 channels, but is processed manually which is slow, inconsistent and incomplete in providing insight. Specifically:

- **No consistent labelling.** If the same problem is reported by multiple employees in various ways, such as "late delivery", "slow shipping" and "delivery took a week", it is not considered to be a single problem.
- **Trends are invisible**. A written summary of a problem is the same if it is reported ten times as a single loud complaint.
- **There is a loss of positive feedback.** Summaries tend to concentrate on complaints whereas what is working well and should be sustained or developed is not reported.
- **No prioritization.** The management is unable to easily distinguish between high severity/recurrence and low severity/occurrence.
- **No traceability.** A summary states ‘customers complain about delivery’ with no indication of comments that support that, meaning management can't verify.
- **Effort scales linearly.** The more additional comments, the more time will be spent reading and writing and no comments will be reused between review cycles.

Current Process (AS-IS Workflow)

The current process is entirely manual and is repeated in full every reporting cycle.

Step	Who	Activity	Output
1	Store / web / app systems	Feedback collected in separate places (survey forms, inbox, website form, app store)	Scattered raw comments
2	Employee	Copies comments into a spreadsheet or document	Single list of comments
3	Employee	Reads each comment and decides in their head what it is about	Informal mental notes
4	Employee	Writes a narrative summary of "main themes"	Word/email summary
5	Employee	Adds suggestions for improvement based on personal judgement	Suggestion list
6	Management	Reads the summary, asks follow-up questions, requests the underlying comments	Questions back to employee
7	Employee	Searches the list again to find evidence	Temporary evidence
8	Management	Decides which changes (if any) to approve	Approved actions

Process Analysis

Characteristic	Steps affected	Why it is a problem
Repetitive	2, 3, 7	Copying and re-reading every comment each cycle, searching for evidence twice
Time-consuming	3, 4, 7	Effort grows with every extra comment, a 300-comment month is ten times the work of a 30-comment month
Difficult to do consistently	3, 4	Labels and themes depend on who reads the comments and on what they read first
Judgement-based	3, 5	Deciding category, severity and whether something is a "trend" is subjective when done by feel
Dependent on management review	6, 8	Management must approve any change, but currently also has to do the evidence-checking work themselves

AI Opportunity

Improve with AI:

- **Classification (step 3).** A language model based on a fixed set of labels is quicker and more consistent than human labelling ad hoc.
- **The detection of trends and counting (step 4)** Deterministic code counts categories and issue tags and has a threshold, eliminating the subjectivity of what "repeated" means.
- **Evidence linking (steps 4, 7).** No longer is there the feedback loop for finding the evidence as each finding comes with the feedback IDs.
- **Drafting recommendations (step 5).** An agent with the business context can suggest viable next-step solutions to solve each identified problem.
- **Report compilation (step 4).** A report-writer agent creates a coherent, structured management report each time it runs.

Keep with a person:

- **Approving operational changes.** The business context and the brief are explicit, AI recommends, management approves. This is enforced by a workflow approval gate in the form of an e-mail.
- **Reviewing classification of mixed, vague or low confidence classifications.** These are placed in a register and presented to management not in secret.
- **Reviewing recommendations not passing Automated QA.** The guardrail agent will not block or auto accept the report after one retry, but instead it will be flagged for additional review.
- **Deciding what a "trend" threshold should be.** The threshold (currently 3) is a business parameter that management can change, not AI.

Part B: Multi-Agent System Design

Agent Design

The system uses six LLM agents with distinct responsibilities, supported by deterministic code nodes for anything that should not be left to a model (counting, thresholds, ID validation, baseline storage). Each agent runs on Google Gemini (gemini-3.5-flash-lite) with a JSON schema output parser so its output is machine-checkable.

Agent	Receives	Does	Passes on
Agent 1 – Classification Agent	One feedback record (id, date, rating, source, text)	Assigns category, sentiment, severity, a fine-grained issue_tag, mixed_signal flag, secondary category/note, confidence, verbatim evidence_quote and rationale. Uses tool get_classification_reference to re-read the official definitions.	One structured classified record
Agent 2 – Trend & Pattern Analyst	Deterministic evidence base: counts, is_pattern flags, evidence lists, deltas vs previous run	Describes recurring negative and positive patterns with example IDs, lists isolated comments, identifies priority concerns, states what changed since last run. Tool: lookup_feedback_by_id.	negative_patterns, positive_patterns, isolated_comments, priority_concerns, change_since_last_run
Agent 3 – Emerging & Anomaly Agent	Same evidence base as Agent 2	Flags non-pattern items that still deserve attention (e.g. single High-severity incidents), summarises the mixed/unclear register, calls out issue tags that are "new" versus the baseline. Tool: lookup_feedback_by_id.	emerging_issues, notable_single_incidents, unclear_feedback_summary

Agent 4 – Business Recommendation Agent	Merged Agent 2 + Agent 3 findings (+ retry_note if a previous attempt failed QA)	Proposes one practical recommendation per priority concern, significant pattern and notable anomaly, plus at least one that reinforces a positive pattern. Every item cites evidence IDs and is marked requires_management_approval. Tool: get_business_context.	recommendations[] with label, evidence IDs, recommendation, expected benefit, priority
Agent 5 – Guardrail / QA Agent	Recommendations + result of the code-based Evidence Integrity Check	Checks that each recommendation follows from its evidence, fits the business context (no major infrastructure first step), uses suggestion language, and introduces no untraceable claims. Fails automatically if any cited ID was invalid.	passed (true/false) and feedback_for_retry
Agent 6 – Report Writer	Compiled report data (evidence counts, findings, recommendations, QA status)	Writes the management report in plain English using only the supplied JSON, in a fixed section structure, showing IDs/counts beside every claim and adding QA warnings where relevant.	Markdown report to HTML then emailed for approval

Supporting deterministic components

Component	Purpose
Loading Sample Feedbacks / Google Sheets Trigger + Normalize Survey Sheet Row	Two intake paths (the supplied dataset and live survey rows) normalized into one schema and merged
Loop + Accumulator	Classifies one record at a time (rate-limit pacing) and collects results in workflow static data
Compute Evidence	Counts categories, issue tags, sentiments and severities; sets is_pattern = count ≥ 3, builds evidence lists and the mixed/unclear register
Load Previous Baseline / Compute Deltas vs Baseline	Compares this run's counts with the last approved run; labels each key new / worsening / improving / unchanged
Evidence Integrity Check	Verifies every feedback_id cited by Agent 4 exists in this run's data
Check Retry Cap / Passed Validation? / Prepare Retry	Sends failed recommendations back to Agent 4 once with the QA feedback; after one retry the report proceeds with a warning
Send Report for Management Approval (Gmail send-and-wait)	Pauses the workflow until a manager clicks Approve or Request Changes
Update Baseline & Audit Log / Log Rejection	Only an approved run becomes the new baseline, and only if its run_timestamp is newer than the baseline already stored, so a late approval of an older run cannot overwrite a newer one. Updates the status sheet with APPROVED or CHANGES REQUESTED option.

Why Multiple Agents?

One can imagine a much faster alternative to the above, asking a model to “read all 30 comments and write a management report” which is quicker to construct, but it doesn't meet the brief in four ways:

- **Consistency.** One-by-one classification against a schema of enumerated labels is much more reliable than asking a model to classify 30 comments in one context and give them all the same labels. The mistakes in one comment do not affect other comments.
- **Judgement and arithmetic separated.** Counting and thresholding are not performed by an LLM, but rather in code. Agent 2 and 3 are informed not to re-compute counts. The one most important thing to keep in mind is that this is the most important protection against any ‘trend being invented by a model'.
- **Checkability**. Each agent has a very specific job and JSON schema, meaning that its output can be validated, then logged for Agent 4 which is then audited independently both by Agent 5 and code. A monolithic prompt creates a single blob to test, which is hard to check.
- **A variety of interpretations of the same information.** Agent 2 searches for repetitions, Agent 3 intentionally searches for that which is not. Having two agents do the same thing can result in the same big themes obscuring the single incidents with high severity.
- **Narrow scope.** Each agent is only given what it requires to do, with only brief prompts and limited risk of hallucination.

Workflow Design (TO-BE)

Feedback moves through the system in eight stages. Stages 2, 4, 5 and 6 are LLM agents and the rest are deterministic.

Stage	Node(s)	What happens
1. Intake	Try me > Loading Sample Feedbacks; Google Sheets Trigger > Normalize Survey Sheet Row; Unified Feedback Intake (Merge, Append)	Dataset records or new survey rows are normalized to feedback_id, date, rating, source, feedback, channel, ingested_via. A run started by the Sheets trigger analyses only the newly added rows; the full-dataset run is started manually.
2. Classification	Reset Accumulator > Loop > Wait 5s > Agent 1 > Accumulate > (loop) > Collect > Aggregate	Each record is classified individually and results are gathered into one list
3. Evidence computation	Compute Evidence > Load Previous Baseline > Compute Deltas vs Baseline	Counts, thresholds, evidence lists, mixed/unclear register, deltas vs last approved run
4. Analysis	Wait 5s > Agent 2 and Agent 3 > Wait for Agents 2 & 3 (Merge) > Merge Trend and Anomaly Insights	Pattern findings and anomaly findings produced from the same evidence base and joined
5. Recommendation	Wait 5s > Agent 4 > Evidence Integrity Check	Recommendations drafted; every cited ID validated in code
6. QA + retry	Wait 5s > Agent 5 > Check Retry Cap > Passed Validation? > (fail) Prepare Retry > Agent 4 again (max 1)	Qualitative check; one automatic correction cycle
7. Report	Compile Management Report Data > Wait 5s > Agent 6 >	Management-readable report assembled and formatted

	Convert Report to HTML > Final Management Report	
8. Human approval	Send Report for Management Approval > Management Approved? > Update Baseline & Audit Log / Log Rejection	Manager approves or requests changes by email; only approval updates the baseline

Human Review

Where	What the person does	Why
Management approval email (mandatory gate)	Reads the report; clicks Approve or Request Changes	Business context states management must approve operational changes. Status sheet records PENDING as the status until approval or rejection. Nothing is recorded as the new baseline until approval.
Mixed or Unclear Feedback section of the report	Reviews feedbacks the system flagged as mixed, vague or "Other"	The system declines to force a label; a person decides whether follow-up with the customer is needed.
"Needs extra review" warning	Reads recommendations with extra scrutiny	Appears only when Agent 5 still failed after one retry; the run proceeded rather than blocking, so a human must weigh the concern.
Trend threshold and business context	Adjusts TREND_MIN_COUNT or the context text	These are business policy settings, not AI decisions.
Rejection log	Follows up on reports where changes were requested	A rejected report is logged with the status (CHANGES REQUESTED) the next run will re-analyses from scratch.

Part C: Feedback Analysis & Business Logic

Classification Approach

Instead of relying on the model to behave, consistency is enforced in three layers:

1. **Prompt.** The nine categories, three sentiments and three severities are all defined in Resource 2 in Agent 1's system message: “Never invent new labels.”
2. **Schema.** The Classification Output Schema contains category, sentiment, severity and confidence fields, all of which are defined by JSON-schema Enums. Any value not found in the list is not considered to be part of the list and the node retries parsing the value, an invented label will never enter the counting phase.
3. **Reference tool.** The get_classification_reference tool provides the official definitions when requested; the agent does not require to memorize the definition, but to check the source.

Finer-grained issue tags: Outside of the category, Agent 1 assigns a short snake_case issue_tag (e.g. late_delivery, delivery_tracking_not_updated, checkout_crash). Two comments only have the same tag if they address the same "problem". This enables the system to distinguish between problems within a category which is important for prioritization.

Comments of more than one concept: The schema includes a primary category, mixed_signal, secondary_category and secondary_note. A comment like “Good quality products, but more discounts would be nice” (F022) belongs to Product Quality / Positive, but not to one or the other category of the issue; it is not forced into one of the categories. Both sides are then exposed: the positive half is reported in the positive register, the pricing half in mixed/unclear register and Agent 3's summary.

Trend Identification

An LLM will not determine whether something is a pattern or not, it will be determined by code:

- Compute Evidence tallies all categories and all issue_tags for the entire run.
- The key is only marked as is_pattern = true if the count of the key equals to TREND_MIN_COUNT = 3. If it is below that, it's an isolated comment.
- There is an evidence array on each trend, with elements: sentiment, severity, evidence_quote and evidence_id along with the count.
- Agent 2 is given instructions: "For every trend with is_pattern = false, list it as an isolated_comment. One comment is one comment, never a trend. Agents 2 and 3 are instructed to not do a recalculation of the counts.
- Compute Deltas vs Baseline will compare the counts of this run with the count in the last approved run, and indicate each count as "new" / "worsening" / "improving" / "unchanged", allowing the report to not only state "delivery is a pattern" but "delivery has grown from 4 to 9 since last review”.

The evidence to support a trend is thus the number, the threshold, the list of feedback IDs and the verbatim quotes, as well as the change compared to the previous baseline.

Mixed or Unclear Feedback

Agent 1 has three explicit escape routes instead of being forced to a single confident answer:

Situation	Agent 1 behavior	Downstream handling	Example from dataset
Two ideas / positive and negative together	mixed_signal = true; secondary_category and secondary_note filled	Enters mixed_or_unclear_register with reason "mixed_signal"; Agent 3 summarizes it; report has a Mixed or Unclear section	F009 "The products are good but I think some prices could be more competitive."
Vague, not enough information	confidence = "low"; best-supported category or "Other"; rationale explains why	Register reason "low_confidence"; never counted as a pattern on its own; flagged for human reading	F017 "Everything was okay. Nothing particularly good or bad."
Does not fit any category	category = "Other"	Register reason "uncategorised_other"; surfaced by Agent 3 as a possible new theme	A comment about parking or opening hours, if received

The rule in Agent 1's prompt is explicit: "Do not guess a specific complaint that isn't stated."

Business Recommendations

There are two separate mechanisms for maintaining the evidence and the context of Urbanmart:

- **Evidence linkage.** Agent 4 is only given the aggregated results from Agent 2 and Agent 3 which have their own feedback IDs. All recommendations need to have an evidence_feedback_ids in its schema. The Evidence Integrity Check then confirms each ID in code; if there is an ID that is not in this run's data, then the check will fail and Agent 5 will refuse to accept the recommendations.
- **Business context.** Resource 3 (2-3 working day delivery promise, 24hour first response, preference for practical low investment fixes, management only decision authority) is included in Agent 4's system message and is also available to Agents 4 & 5 via the tool get_business_context. Agent 4 "must fit" each recommendation and Agent 5 checks it independently.
- **Suggestion language and approval flag.** All recommendations should include “Consider…/Recommend…” language and should be requires_management_approval = true. Agent 5 is not validated if any of these are absent.
- **Positive reinforcement**. It is not complaint-only as Agent 4 will need to provide at least one recommendation that maintains or expands a positive pattern (e.g. recognize helpful staff in the store).

Evidence

Level	What is shown	Where it comes from
Individual comment	feedback_id, verbatim evidence_quote (< 12 words), rationale	Agent 1
Pattern	Label, count, is_pattern flag, 2–4 example feedback IDs	Compute Evidence + Agent 2
Anomaly / single incident	feedback_id, category, severity, note	Agent 3
Recommendation	Related label, evidence_feedback_ids, expected benefit, priority	Agent 4 + Evidence Integrity Check
Run-level	total_feedback_analysed, sentiment_counts, severity_counts, deltas vs baseline	Compute Evidence / Compute Deltas
Report	Every section shows IDs or counts beside each claim; QA status lines	Agent 6 (instructed to use only the supplied JSON)

Part D: Build the Working System

Platform and tool choices

Choice	Reason
n8n	Visual multi-agent orchestration with native AI Agent, Structured Output Parser, Code, Loop, Merge and Wait nodes; every execution is logged and inspectable node-by-node, which is ideal for demonstrating and testing.
Google Gemini gemini-3.5-flash-lite	Low cost and fast, adequate for structured classification and summarization when paired with schema enforcement. Six separate model nodes let each agent be tuned independently.
Structured Output Parser on every agent	Turns free text into validated JSON so code can count, check and merge outputs safely.
Google Sheets Trigger	Realistic live intake path: new survey rows (e.g. from Google Forms) enter the same pipeline as the supplied dataset.
Gmail send-and-wait	Simple, auditable human-approval gate with Approve / Request Changes buttons that pause the workflow.
Workflow static data	Stores the previous approved baseline and the per-run classification accumulator without an external database.
Google sheets	Creates and updates the status sheet for these columns: run_id, run_timestamp, total_feedback, status, reviewed_at.

Meeting the minimum criteria

The system meets all the minimum criteria by using 6 specialized agents and deterministic code nodes. The six agents (classification, trend analysis, anomaly detection, recommendation, quality assurance and report writing) have different purposes, and the code nodes around them deal with all things that should not be delivered by a language model like counting, thresholding, identifier validation, etc.

Use of the three resources

The feedback resource (Feedback Classification Reference) is available to the agent when the dataset is loaded, and is also passed to Agent 1 in the prompt;
the classification reference is embedded in the prompt that Agent 1 is given;
the UrbanMart Business Context is embedded in the prompt given to Agent 4, and is also passed to Agents 4 and 5 via the get_business_context tool.

Complete dataset and consistent labels

The complete data set is analyzed since every record is processed by the loop node and then aggregated by the aggregate node, and the value of the total_feedback_analysed is returned which then the management can verify that no records were missed.
An enumerated JSON schema is in place for Agent 1's output and referenced using the reference tool, and therefore an invented label cannot pass validation.

Patterns vs individual comments

The difference between genuine pattern and an individual comment is not based on an agent but it is based on the code: a category or an issue tag is considered as a pattern only if its count is equal or greater than the Trend_MIN_COUNT (which is set to 3) threshold. All entries below that threshold will be treated as separate individual comments.

Positive feedback, priorities and evidence

Positive feedback is handled as a first-class-outcome, and is displayed in a dedicated positive_patterns array, as a required, positive-reinforcing recommendation from Agent 4, and in the Positive Patterns section of the final report.
Priority concerns and notable_single_incidents and emerging_issues are produced from two fronts: from Agent 2 from the recurring patterns, and from Agent 3 from items that do not yet exist as patterns, but are still of interest to be addressed.
Each recommended identifier must include evidence_feedback_ids and the Evidence Integrity Check checks each identifier against the evidence in this run; it may not be based on a comment that is not present.

Mixed, unclear and strange feedback

Mixed, unclear, and strange feedback is dealt with without the adding of information: If Agent 1 sets mixed_signal, the agent puts the lowest confidence rating, or if the best fit is not clear, the agent puts Other, and all such items are collected into a register that Agent 3 summarizes for human reading with an explicit "do not guess" rule in the classification prompt.

Human approval and reacting to new feedback

Business changes are not yet automated, as the report is sent via a Gmail approval chain, each recommendation is marked as "every recommendation is flagged which require_management_approval" and only the saved baseline is updated after approved by a manager.

The system also reacts to new feedback that affects the picture – each approved run is stored as a baseline, the subsequent run calculates the deltas labeled new, worsening, improving or unchanged, and the Google Sheets trigger makes new rows enter the pipeline.

Finally, no count, pattern and recommendations are hard-coded; they're based on whatever data you feed into a particular run.

Rules given to each agent

- Agent 1 will only use information from the feedback as written, and will not add to or make up information that was not included in the feedback; its evidence quote must be exactly as stated in the feedback, with a maximum of 12 words. It must return the feedback identifier unchanged.
- Agent 2 is informed that it is not allowed to re-calculate counts, it is not allowed to contradict counts, that the one comment cannot be a trend and that every finding requires feedback identifiers.
- Agent 3 has been told to ignore the bulk of the recurring patterns and only report the unusual, the new and the edge cases and that one single situation of high severity should be reported as a single incident, not a trend.

- Agent 4 must provide evidence for every recommendation, fit the recommendation to the business context, include suggestion language, be indicated as requiring management approval and include at least one recommendation that supports and reinforces a positive pattern.
- Agent 5 will automatically fail validation if any integrity check fails and will check evidence linkage, business fit, suggestion language and no untraceable claims; provides succinct feedback for retries, if necessary.
- Agent 6 can only use the JSON payload he is given, and can only add an identifier or a count next to each claim, with a warning line when he had to use quality assurance after he hit the retry count.

Part E: Testing & Validation

Tests were run on the local n8n instance using the "Try me" trigger (full dataset) and the Google Sheets trigger (new & updated rows).

Testing Records

Test	What you changed / tested	Expected behavior	Actual behavior	Status
Test 1: Complete data	Ran "Try me" with all 30 records end to end	All 30 classified; Delivery, Staff Behavior, Pricing, Product Availability and Website/App reach is_pattern; positive Staff Behaviour pattern reported; report emailed for approval	• **Total Feedback Analyzed:** 30 • **Sentiment Breakdown:** Positive: 10 | Neutral: 3 | Negative: 17 • **Severity Breakdown:** Low: 3 | Medium: 14 | High: 3 • **Approval Status:** PENDING_MANAGEMENT_REVIEW • **Report received for approval**	**Passed**

Test 2: Mixed feedback	Sent F022 "Good quality products, but more discounts would be nice." and F009	mixed_signal = true; primary Product Quality/Positive, secondary Pricing; item appears in Mixed or Unclear section and in positive counts	{
			"category": "Product Quality",
			"issue_tag": "good_product_quality",
			"mixed_signal": true,
			"secondary_category": "Pricing",
			"secondary_note": "Customer also desires more discounts.",
			"sentiment": "Positive",
			"severity": "N/A",
			"confidence": "high",
			"evidence_quote": "Good quality products",
			"rationale": "The customer expresses positive sentiment about product quality ('Good quality products') while also expressing a desire for better pricing/discounts ('more discounts would be nice'), making it a mixed signal touching both Product Quality and Pricing."
			}
				Passed

Test 3: Unclear feedback	Sent F017 "Everything was okay. Nothing particularly good or bad." and a new vague row "Meh."	sentiment Neutral, confidence low, category Other or best guess with rationale; no specific complaint invented; not counted as a pattern	{ "mixed_signal": false, "sentiment": "Neutral", "severity": "N/A", "confidence": "high", "evidence_quote": "Nothing particularly good or bad.", "rationale": "The customer expresses complete indifference	**Passed**

			with no specific issue or praise mentioned across any category, fitting the neutral Other classification." }	

Test 4: New or unusual feedback	Added via Google Sheet: "The car park was full and I could not find a space." (no matching category)	category Other; Agent 3 lists it under emerging_issues / unclear summary; Agent 2 does not treat it as a trend	"emerging_issues": [
			{
			"label": "Store Experience - car_park_full",
			"reason": "Both the category 'Store Experience' and issue tag 'car_park_full' have a status of 'new' compared to the previous baseline (absent previously, count of 1 now).",
			"evidence_feedback_ids": [
			"SUR-2"
			]
			}
			],
				Passed

Test 5: Changing trend	Approved a baseline run, then added 4 new checkout-crash comments to the sheet and re-ran	checkout_crash count rises past 3 > becomes is_pattern; delta status "worsening"; Agent 2 notes the change; Agent 4 adds a Website/App recommendation	{
			"issue_tag": "checkout_crash",
			"count": 4,
			"is_pattern": true
			"category": "Website / App"
			}
				Passed

Additional defects found during build

- Timestamp parsing (Google Forms). The first ten characters of Timestamp are "9/18/2026 14:32:10", which is an invalid date. Fixed by parsing with new Date(...).toISOString() is never used because it always returns YYYY-MM-DD.
- The missing of Drive scope on the Sheets Trigger. The Gmail-only credential was not sufficient to include the Google Drive API and Drive scope, which was necessary for “Row Added or Updated” mode. The credential is re-authorized with the additional scope and the trigger is updated to "Row Added", meaning that pipeline will only re-run when a row is added and won't be re-run when the cell is edited, thus preventing multi-counts.
- Max output tokens. Agent 1 cut off the JSON in longer comments, since it didn't set an output limit, the structured parser broke. It was solved by increasing the maxOutputTokens on the model node to 2048.
- Rate limiting. The agents returned 429 errors when they ran back to back with Gemini. Before each agent, a 5s Wait node was inserted and each agent was given a maximum of 3 retries with a back-off of 8s.
- Direct Markdown to email. The report writer's Markdown was sent as raw asterisks and hashes in the inbox. A "Convert Report to HTML" code node was added in front of the "Send and wait" node to the gmail node so that the approval email appears as a formatted report.
- Merging of agent outputs in parallel. Agent 2 completed before Agent 3, and Agent 2's code node combined with Agent 3's code node fired on completion of Agent 2, and errors out. A Merge node (Append, two inputs) was added so that the workflow could wait for both agents to be finished before it starts to combine the results.
- Accumulator not zeroed between runs. The static-data list used to store Agent 1 results was used again on each subsequent iteration, so the second iteration also included the results of the first iteration. By adding a “Reset Classification Accumulator” at the beginning of classification, each run will be started fresh, and only record the current data.

Part F: Final System Output

The final output is an HTML formatted email with the UrbanMart Customer Feedback Report, Approve and Request Changes buttons. The report will be generated by Agent 6 in a fixed format so that management can read the report without having to visit any agent:

- Overview – total number of feedbacks analyzed, number of feedbacks with positive sentiment, number of feedbacks with negative sentiment, range of report dates.
- Positive Patterns – positive experiences repeated with counts and IDs – 5.
- Recurring Issues / Priority Concerns – patterns that met the threshold, were ranked, and there was justification.
- Emerging or Unusual Issues – new tags versus baseline, single high severity incidents
- Isolated / One-off Comments – comments which have been listed below the threshold and thus are not lost.
- Mixed or Unclear Feedback – register of mixed_signal / low-confidence / Other items for human reading
- Recommendations (with evidence) – each with related label, evidence IDs, expected benefit, priority and approval flag
- What Changed since Last Run – deltas compared to previous approved baseline or "first run" note.
- Next Steps for Management – what needs a decision, what needs further investigation

Part G: Business Review

Business Impact

Area	Current (AS-IS)	With the system (TO-BE)
Time	Every comment read and summarized by hand each cycle which helps effort grow linearly	Classification and counting are automatic; the employee's role shrinks to reviewing the register and the report
Consistency	Labels vary by reader and by day	One enumerated label set enforced by schema; same comment = same label
Visibility of repeats	Hard to tell one loud complaint from a recurring problem	Explicit counts, a stated threshold, and deltas versus the last review
Use of positive feedback	Usually omitted	Positive patterns reported and reinforced by at least one recommendation
Evidence	Management asks "show me"; employee searches again	Every claim carries feedback IDs; a lookup tool verifies quotes
Management review	Reads narrative, decides with limited evidence	Reads a structured report, approves or requests changes with one click; decision remains theirs
Scalability	A 300-comment month is 10× the work	Marginal cost per comment is one small LLM call

Limitations

- Classification errors. A small model may confuse sarcasm, idioms/comments in another language. Schema enforcement will ensure a valid label but will not guarantee a correct label. Mitigation: confidence field, mixed/unclear register, human review of the report.
- Threshold sensitivity. You can use TREND_MIN_COUNT = 3 for a set of 30 records. It would over-flag with 3000 records, and under-flag with 10. The threshold should scale with volume (e.g. a percentage) in production.
- Issue-tag drift. Sometimes an issue can be tagged with two different issue_tags, due to the fact that they are free-form slugs, which will split up a count (late_delivery vs delivery_late). This can be decreased by implementing a tag-normalization step or a controlled tag list.
- Biased or incomplete samples. Customers who are extremely unhappy or extremely happy over-respond to the survey. Surveys of storing dominate this data set. The system only reports source numbers; sampling bias cannot be corrected.
- Business constraints not in the context. Agent 4 has only a knowledge of Resource 3. It doesn't care about staffing levels, supplier contracts, budget cycles or legal constraints, and there may be some "practical" suggestions which are not feasible. That is why it's still under management's approval.
- Baseline is single-slot. The system reports change since the last review, not the longer the trend line, only the last approved run is stored.

Future Improvement

- Persistent storage for runs and classifications. Use a database (Postgres or a Google Sheet “Classified” tab) to replace static data in the workflow so trends can be charted over many cycles and audited.
- Controlled vocabulary of tags of issues. Keep an ever-growing list of approved issue_tags. Agent 1 first considers this list, and only proposes new tags when nothing else fits. New tags are reviewed by a human.
- Let management set the pattern threshold as a percentage of the run plus an absolute minimum.
- Low-confidence items human-in-the-loop. route low confidence classifications to a reviewer for review before they enter the counts, not just after reporting.
- Add triggers for Gmail, website form & app-store review. All normalized to the same schema, in addition to the Google Sheet.
- Maintain a human labeled gold set (initially the 30 records) and measure Agent 1 accuracy after each prompt or model change.
- Publish counts and deltas to a simple dashboard so management can see the picture between report cycles.

Final Presentation

Link to final presentation